

Construcción de modelos base

introducción

- **Lo que hará tu compañero (Tarea 1.5):** Analizará el *DataFrame* (`df_relacional`) que dejaste listo en la Tarea 1.4, combinándolo con el vector de propiedades nativas de los artículos. El objetivo de esta fase es aplicar tests estadísticos para comprender la utilidad de las métricas relacionales extraídas y seleccionar las características más relevantes para **ayudar al modelado y evitar el sobreajuste** de los datos de entrenamiento.
- **Lo que puedes ir avanzando tú (Tarea 2.1):** Dado que ya tienes la estructura de los datos consolidada, puedes empezar a programar toda la "tubería" de *Machine Learning* utilizando la librería *Scikit-Learn*. Puedes alimentar temporalmente tus scripts con todas las características disponibles (o un subconjunto de prueba) mientras tu compañero determina cuáles son las definitivas.

En concreto, para ir adelantando la construcción de los modelos (Tarea 2.1), puedes enfocarte en dejar programado lo siguiente:

- 1. **La partición de los datos:** Puedes escribir el código para dividir vuestro conjunto de ejemplos en subconjuntos de entrenamiento y prueba. Para ello, el contenido teórico recomienda aplicar técnicas como la validación por retención (*holdout validation*) o la validación cruzada con *k* pliegues (*k-fold cross validation*).
- 2. **La inicialización de los algoritmos:** El trabajo exige construir al menos tres modelos de clasificación estándar. Puedes ir importando y configurando los métodos propuestos en la teoría, tales como **k-vecinos más cercanos (kNN)**, **árboles de clasificación**, modelos probabilísticos como **Naive Bayes**, o **redes neuronales** multicapa.
- 3. **La configuración de la búsqueda de hiper-parámetros:** Puedes dejar preparada la estructura de código para realizar una **búsqueda en rejilla** (*grid search*) que evalúe diferentes combinaciones de hiper-parámetros para cada modelo.
- 4. **Las métricas de rendimiento:** Programar el código para extraer la matriz de confusión y calcular medidas como la tasa de acierto (*accuracy*) o la tasa de error, necesarias para estimar la capacidad de generalización.

El punto de integración: En el momento en que tu compañero finalice el análisis de correlaciones (Tarea 1.5) y te indique exactamente **qué características concretas debéis conservar** (por ejemplo, si decide que el *clustering* no aporta información útil o

si filtra ciertas palabras del texto), solo tendrás que modificar las columnas de entrada en los cuadernos que ya has preparado y ejecutar los entrenamientos.

¿Qué necesito de las tareas anteriores para avanzar en esta fase?

Para poder empezar a programar la **Tarea 2.1 (Entrenamiento de modelos)**, necesito traerme las estructuras de datos fundamentales que representan las variables de entrada (X) y la variable objetivo (y).

De la Tarea 1.3 (Análisis Exploratorio): Necesito el código que lee el archivo `cora.content` y separa la información, ya que ahí residen las propiedades nativas y lo que el modelo debe predecir:

- **Las etiquetas (`labels`):** Será la variable a predecir (y). Contiene las 7 clases temáticas de los artículos.
- **Las características nativas (`features`):** Es la matriz con las 1433 columnas de palabras en formato binario.
- **Los identificadores (`paper_ids`):** Son vitales para usar como índice y asegurar de que al cruzar los datos no se desordenen las filas.

De la Tarea 1.4 (Características Relacionales): Necesito el resultado final de la extracción topológica que hice sobre el grafo `G` :

- **El DataFrame `df_relacional` :** Este contiene las cuatro nuevas columnas que calculé: `centralidad_grado` , `centralidad_betweenness` , `coeficiente_clustering` y `comunidad_louvain` indexadas por el `paper_id` .

¿Cómo integrarlo para la Tarea 2.1?

Para entrenar los modelos en Scikit-Learn (como kNN, Árboles de Decisión, etc.), necesito unir ambas fuentes de información en un único conjunto de datos. El paso lógico al inicio del cuaderno para la Tarea 2.1 es:

1. **Concatenar los datos de entrada (X):** Unir las 1433 columnas de `features` con las 4 columnas de `df_relacional` .
2. **Definir la salida (y):** Asignar las `labels` .
3. **Dividir los datos:** Aplicar una función como `train_test_split` de Scikit-Learn para separar este gran conjunto en datos de entrenamiento y datos de prueba.

Consejo organizativo: Como recordatorio de nuestra conversación anterior sobre mantener los cuadernos limpios, tienes dos opciones prácticas para no tener que

volver a calcular todo el grafo de NetworkX cada vez que abras la Tarea 2.1:

- **Opción A:** Al final del cuaderno de la Tarea 1.4, concatena `features` y `df_relacional`, y exporta el resultado usando `df_completo.to_csv('cora_features_completas.csv')`. Así, en tu Tarea 2.1 solo tendrás que hacer un simple `pd.read_csv()`.
- **Opción B:** Copia el bloque de importaciones, la carga de los DataFrames y el código de NetworkX que genera `df_relacional` al principio de tu nuevo cuaderno para que se ejecute en memoria.

Implementación de la Tarea 2.1

En esta sección vamos a construir modelos base de clasificación para Cora usando Scikit-learn, siguiendo una metodología reproducible:

1. Preparar los datos de entrada y salida.
2. Separar entrenamiento y prueba con estratificación.
3. Ajustar hiperparámetros con validación cruzada.
4. Evaluar en test con métricas globales y por clase.

```
In [10]: import warnings
from pathlib import Path

import networkx as nx
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_fscore_support,
)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
pd.set_option("display.max_colwidth", 120)
```

Carga de datos (opción A con fallback automático)

Primero buscamos un CSV consolidado (opción A). Si no existe, construimos el dataset combinado a partir de:

- Atributos nativos de cora.content.
- Estructura relacional en cora.cites.

Este enfoque permite ejecutar el notebook de forma autónoma sin depender de pasos manuales previos.

```
In [2]: def cargar_datos_cora(base_dir: Path):
    csv_candidates = [
        base_dir / "data" / "cora_features_completas.csv",
        base_dir / "data" / "cora" / "cora_features_completas.csv",
        base_dir / "notebooks" / "cora_features_completas.csv",
    ]

    for csv_path in csv_candidates:
        if csv_path.exists():
            df = pd.read_csv(csv_path)
            if "paper_id" in df.columns:
                df = df.set_index("paper_id")

            posibles_label = ["class_label", "label", "target", "y"]
            label_col = next((col for col in posibles_label if col in df.columns), None)
            if label_col is None:
                raise ValueError(
                    f"Se encontró {csv_path}, pero no contiene ninguna columna válida"
                )

            y = df[label_col].copy()
            X = df.drop(columns=[label_col])
            return X, y, f"CSV consolidado: {csv_path}"

    content_path = base_dir / "data" / "cora" / "cora.content"
    cites_path = base_dir / "data" / "cora" / "cora.cites"

    if not content_path.exists() or not cites_path.exists():
        raise FileNotFoundError("No se encontró ni CSV consolidado ni archivos de datos")

    df_content = pd.read_csv(content_path, sep="\t", header=None)
    paper_ids = df_content.iloc[:, 0].astype(int)
    y = df_content.iloc[:, -1].astype(str)

    feature_cols = [f"word_{i}" for i in range(df_content.shape[1] - 2)]
    X_nativo = pd.DataFrame(df_content.iloc[:, 1:-1].values, index=paper_ids)
    X_nativo.index.name = "paper_id"
    y.index = X_nativo.index
    y.name = "class_label"

    df_cites = pd.read_csv(cites_path, sep="\t", header=None, names=["source", "target"])
    df_cites = df_cites.astype(int)
```

```

G = nx.from_pandas_edgelist(df_cites, source="source", target="target",
G.add_nodes_from(X_nativo.index.tolist())

degree = nx.degree_centrality(G)
betweenness = nx.betweenness_centrality(G)
clustering = nx.clustering(G)

comunidades = nx.community.louvain_communities(G, seed=RANDOM_STATE)
map_comunidad = {}
for id_comunidad, nodos in enumerate(comunidades):
    for nodo in nodos:
        map_comunidad[nodo] = id_comunidad

X_rel = pd.DataFrame(
    {
        "centralidad_grado": pd.Series(degree),
        "centralidad_betweenness": pd.Series(betweenness),
        "coeficiente_clustering": pd.Series(clustering),
        "comunidad_louvain": pd.Series(map_comunidad),
    }
)
X_rel.index.name = "paper_id"

X = X_nativo.join(X_rel, how="left").fillna(0)
return X, y, "Reconstruido desde cora.content + cora.cites"

base_dir = Path.cwd().resolve().parent if Path.cwd().name == "notebooks" else
X, y, fuente_datos = cargar_datos_cora(base_dir)

print(f"Fuente de datos: {fuente_datos}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print("Distribución de clases (top 7):")
print(y.value_counts().head(7))

```

Fuente de datos: Reconstruido desde cora.content + cora.cites

X shape: (2708, 1437)

y shape: (2708,)

Distribución de clases (top 7):

class_label	
Neural_Networks	818
Probabilistic_Methods	426
Genetic_Algorithms	418
Theory	351
Case_Based	298
Reinforcement_Learning	217
Rule_Learning	180

Name: count, dtype: int64

Separación de entrenamiento y prueba

Dividimos el dataset en train/test con estratificación para conservar la proporción de clases en ambos subconjuntos. Esto evita sesgos al evaluar modelos multiclase con

clases desbalanceadas.

```
In [3]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=RANDOM_STATE,
        stratify=y,
    )

print(f"Train shape: {X_train.shape}")
print(f"Test shape: {X_test.shape}")
print("\nDistribución de clases en train (top 7):")
print(y_train.value_counts(normalize=True).head(7))
print("\nDistribución de clases en test (top 7):")
print(y_test.value_counts(normalize=True).head(7))
```

Train shape: (2166, 1437)

Test shape: (542, 1437)

Distribución de clases en train (top 7):

class_label	
Neural_Networks	0.301939
Probabilistic_Methods	0.157433
Genetic_Algorithms	0.154201
Theory	0.129732
Case_Based	0.109880
Reinforcement_Learning	0.080332
Rule_Learning	0.066482

Name: proportion, dtype: float64

Distribución de clases en test (top 7):

class_label	
Neural_Networks	0.302583
Probabilistic_Methods	0.156827
Genetic_Algorithms	0.154982
Theory	0.129151
Case_Based	0.110701
Reinforcement_Learning	0.079336
Rule_Learning	0.066421

Name: proportion, dtype: float64

Modelos base y espacios de búsqueda

Entrenaremos cuatro modelos base:

- kNN
- Árbol de Decisión
- Naive Bayes Gaussiano
- MLP (red neuronal multicapa)

Para cada uno definimos un grid razonable de hiperparámetros y usaremos validación cruzada para seleccionar la mejor configuración según F1 macro.

```

In [6]: modelos = {
    "kNN": Pipeline(
        [
            ("scaler", StandardScaler()),
            ("clf", KNeighborsClassifier()),
        ]
    ),
    "ArbolDecision": Pipeline(
        [
            ("clf", DecisionTreeClassifier(random_state=RANDOM_STATE)),
        ]
    ),
    "NaiveBayes": Pipeline(
        [
            ("scaler", StandardScaler()),
            ("clf", GaussianNB()),
        ]
    ),
    "MLP": Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "clf",
                MLPClassifier(
                    max_iter=300,
                    early_stopping=False,
                    random_state=RANDOM_STATE,
                ),
            ),
        ]
    ),
}

grids = {
    "kNN": {
        "clf__n_neighbors": [3, 5, 7, 11],
        "clf__weights": ["uniform", "distance"],
        "clf__p": [1, 2],
    },
    "ArbolDecision": {
        "clf__criterion": ["gini", "entropy"],
        "clf__max_depth": [None, 10, 20, 30],
        "clf__min_samples_split": [2, 5, 10],
    },
    "NaiveBayes": {
        "clf__var_smoothing": np.logspace(-11, -7, 5),
    },
    "MLP": {
        "clf__hidden_layer_sizes": [(64,), (128,), (64, 32)],
        "clf__alpha": [1e-4, 1e-3],
        "clf__learning_rate_init": [1e-3, 1e-2],
    },
}

print("Modelos definidos:", list(modelos.keys()))

```

Modelos definidos: ['kNN', 'ArbolDecision', 'NaiveBayes', 'MLP']

En el bloque anterior se definen dos piezas clave del experimento: `modelos` y `grids`.

- `modelos` contiene cuatro *pipelines* de Scikit-Learn (kNN, Árbol de Decisión, Naive Bayes y MLP), cada uno con su preprocesado y su clasificador. Esto asegura que, durante validación cruzada, el escalado (`StandardScaler`) se aplique correctamente dentro de cada *fold*, evitando fugas de información.
- `grids` define el espacio de hiperparámetros que se va a explorar con `GridSearchCV` para cada modelo. Por ejemplo, en kNN se prueban distintos vecinos, ponderaciones y métricas de distancia; en Árbol se ajustan criterio y profundidad; en Naive Bayes se calibra `var_smoothing`; y en MLP se testean arquitectura, regularización y tasa de aprendizaje. La idea es comparar configuraciones de forma sistemática y reproducible, no quedarse con valores por defecto.
- La métrica objetivo indicada es **F1 macro**, adecuada para clasificación multiclase como Cora. Se calcula como el promedio no ponderado del F1 de cada clase, es decir, todas las clases pesan igual aunque haya desbalance.

La métrica **F1 macro** se construye así:

$$F1_i = \frac{2 \cdot P_i \cdot R_i}{P_i + R_i} \quad \text{y} \quad F1_{macro} = \frac{1}{K} \sum_{i=1}^K F1_i$$

Significado de cada símbolo:

1. i : índice de la clase (por ejemplo, clase 1, clase 2, etc.).
2. K : número total de clases.
3. P_i : *precision* de la clase i .

Es la proporción de predicciones de la clase i que eran correctas.

$$P_i = \frac{TP_i}{TP_i + FP_i}$$

4. R_i : *recall* (sensibilidad) de la clase i .

Es la proporción de ejemplos reales de la clase i que el modelo detectó.

$$R_i = \frac{TP_i}{TP_i + FN_i}$$

5. $F1_i$: F1 de la clase i , que combina P_i y R_i con media armónica.
6. $F1_{macro}$: promedio simple de los $F1_i$ de todas las clases (todas pesan igual).

Y en las fórmulas de precisión/recall:

1. TP_i (*true positives*): aciertos en clase i .
2. FP_i (*false positives*): predijo clase i , pero no era i .
3. FN_i (*false negatives*): era clase i , pero el modelo no la predijo como i .

Idea clave: **macro** significa que cada clase cuenta por igual, aunque haya clases con pocos ejemplos. Esto es crucial en datasets desbalanceados como Cora, donde algunas clases pueden ser minoritarias. Esto fuerza al modelo a **rendir bien en clases minoritarias, no solo en las más frecuentes**.

Búsqueda de hiperparámetros con validación cruzada

Para cada modelo aplicamos GridSearchCV con validación cruzada estratificada de 5 folds. El criterio principal de selección será F1 macro, porque pondera por igual todas las clases.

Nota

La **validación cruzada estratificada de 5 folds** es una técnica para estimar el rendimiento real de un modelo sin depender de una sola partición de entrenamiento/prueba. El conjunto de entrenamiento se divide en 5 partes (folds) de tamaño similar. En cada iteración, el modelo se entrena con 4 partes y se valida con la parte restante, repitiendo el proceso 5 veces hasta que cada fold haya sido usado una vez como validación.

La palabra **estratificada** significa que en cada fold se conserva, aproximadamente, la misma proporción de clases que existe en el conjunto original. Esto es especialmente importante en problemas multiclase y desbalanceados (como Cora), porque evita que una clase minoritaria quede mal representada en alguna partición y distorsione la evaluación.

Al final, se promedian las métricas de las 5 iteraciones (por ejemplo, F1 macro). Ese promedio es más estable y fiable que evaluar con una sola partición.

En términos prácticos: reduce la varianza de la evaluación, ayuda a comparar modelos de forma más justa y mejora la selección de hiperparámetros en `GridSearchCV`.

```
In [7]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

mejores_modelos = {}
resumen_busqueda = []

for nombre, pipeline in modelos.items():
    print(f"\nEntrenando {nombre}...")
    search = GridSearchCV(
        estimator=pipeline,
        param_grid=grids[nombre],
        scoring="f1_macro",
```

```

        cv=cv,
        n_jobs=-1,
        verbose=0,
    )
    search.fit(X_train, y_train)

    mejores_modelos[nombre] = search.best_estimator_
    resumen_busqueda.append(
        {
            "modelo": nombre,
            "best_cv_f1_macro": search.best_score_,
            "best_params": search.best_params_,
        }
    )

resumen_cv = pd.DataFrame(resumen_busqueda).sort_values("best_cv_f1_macro",
resumen_cv

```

Entrenando kNN...

Entrenando ArbolDecision...

Entrenando NaiveBayes...

Entrenando MLP...

Out[7]:

	modelo	best_cv_f1_macro	best_params
1	ArbolDecision	0.747734	{'clf__criterion': 'gini', 'clf__max_depth': 20, 'clf__min_samples_split': 5}
3	MLP	0.705691	{'clf__alpha': 0.001, 'clf__hidden_layer_sizes': (64,), 'clf__learning_rate_init': 0.01}
2	NaiveBayes	0.408144	{'clf__var_smoothing': 1e-08}
0	kNN	0.389740	{'clf__n_neighbors': 3, 'clf__p': 2, 'clf__weights': 'distance'}

Evaluación final en el conjunto de prueba

Una vez fijado el mejor conjunto de hiperparámetros por validación cruzada, evaluamos en test para estimar rendimiento fuera de muestra.

Métricas reportadas:

- Accuracy
- Precision macro
- Recall macro
- F1 macro
- Reporte por clase

```

In [8]: resultados_test = []
reportes_clase = {}
matrices_confusion = {}

for nombre, modelo in mejores_modelos.items():
    y_pred = modelo.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision, recall, f1_macro, _ = precision_recall_fscore_support(
        y_test, y_pred, average="macro", zero_division=0
    )

    resultados_test.append(
        {
            "modelo": nombre,
            "accuracy": accuracy,
            "precision_macro": precision,
            "recall_macro": recall,
            "f1_macro": f1_macro,
        }
    )

    reportes_clase[nombre] = classification_report(y_test, y_pred, zero_division=0)
    matrices_confusion[nombre] = confusion_matrix(y_test, y_pred, labels=np.unique(y_test))

resumen_test = pd.DataFrame(resultados_test).sort_values("f1_macro", ascending=False)
resumen_test

```

```

Out[8]:

```

	modelo	accuracy	precision_macro	recall_macro	f1_macro
1	ArbolDecision	0.771218	0.762757	0.760281	0.758796
3	MLP	0.730627	0.721221	0.717874	0.717067
2	NaiveBayes	0.452030	0.437962	0.422300	0.428077
0	kNN	0.422509	0.469593	0.406266	0.416785

Selección del mejor modelo y análisis detallado

Tomamos como mejor modelo el que obtiene mayor F1 macro en test. Después mostramos:

- Reporte de clasificación por clase.
- Matriz de confusión para observar dónde se concentran los errores.

```

In [9]: import matplotlib.pyplot as plt

mejor_nombre = resumen_test.iloc[0]["modelo"]
print(f"Mejor modelo en test (según F1 macro): {mejor_nombre}")

print("\nReporte por clase:\n")
print(reportes_clase[mejor_nombre])

```

```

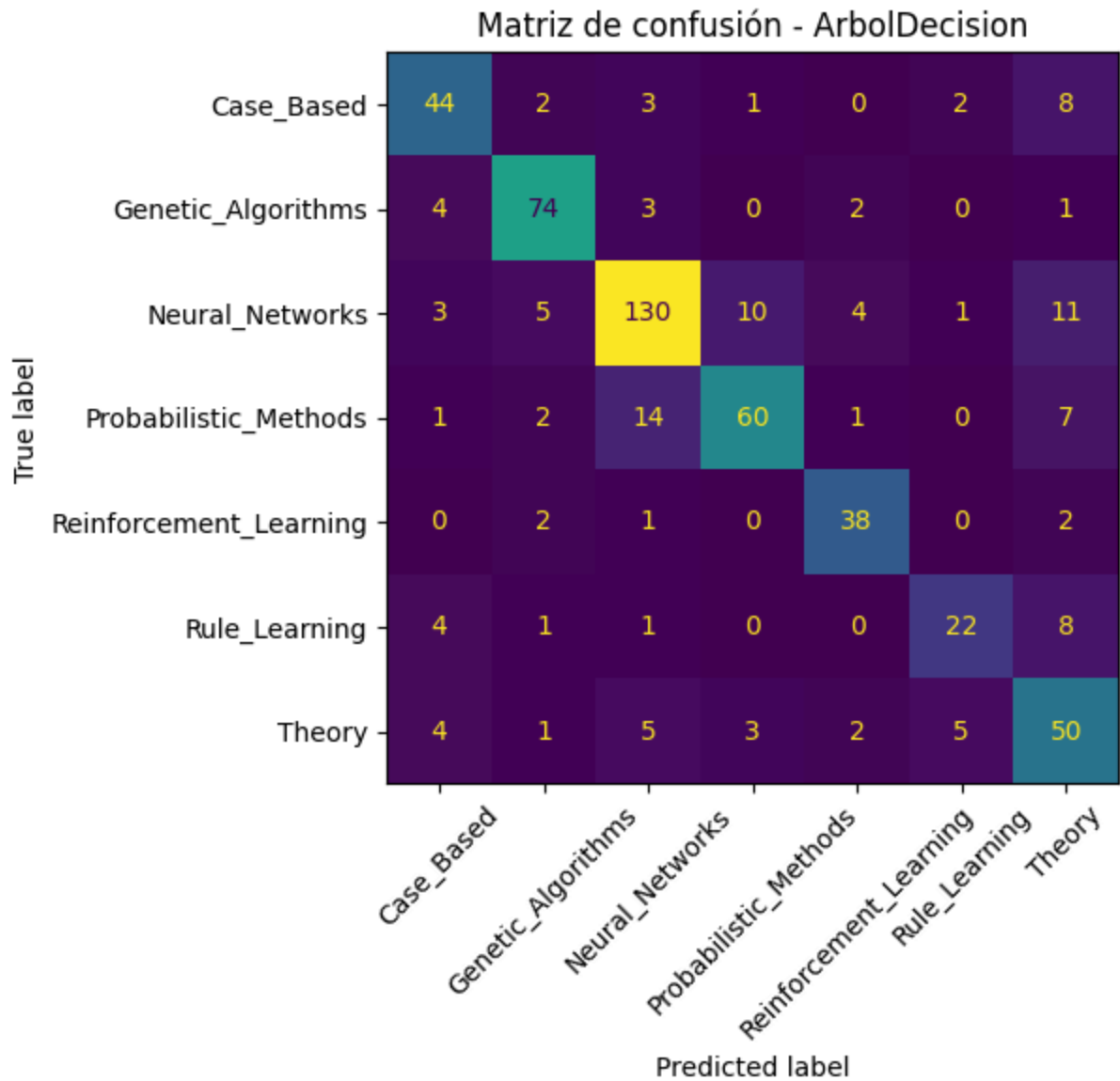
labels_ordenados = np.unique(y_test)
fig, ax = plt.subplots(figsize=(8, 6))
ConfusionMatrixDisplay(
    confusion_matrix=matrices_confusion[mejor_nombre],
    display_labels=labels_ordenados,
).plot(ax=ax, xticks_rotation=45, colorbar=False)
ax.set_title(f"Matriz de confusión - {mejor_nombre}")
plt.tight_layout()
plt.show()

```

Mejor modelo en test (según F1 macro): ArbolDecision

Reporte por clase:

	precision	recall	f1-score	support
Case_Based	0.73	0.73	0.73	60
Genetic_Algorithms	0.85	0.88	0.87	84
Neural_Networks	0.83	0.79	0.81	164
Probabilistic_Methods	0.81	0.71	0.75	85
Reinforcement_Learning	0.81	0.88	0.84	43
Rule_Learning	0.73	0.61	0.67	36
Theory	0.57	0.71	0.64	70
accuracy			0.77	542
macro avg	0.76	0.76	0.76	542
weighted avg	0.78	0.77	0.77	542



Persistencia opcional de resultados y modelo (trazabilidad)

Esta celda permite guardar los artefactos del experimento (métricas, reporte, modelo y metadatos) en una carpeta con timestamp.

- Por defecto está desactivada (`GUARDAR_ARTEFACTOS = False`).
- Si la activas en `True` , se crea un directorio en `artifacts/tarea_2_1/<timestamp>/` con todo lo necesario para reproducir y auditar el resultado.

```
In [13]: # Persistencia opcional para trazabilidad de experimentos (activar bajo demanda)
from datetime import datetime
import json
from joblib import dump
import matplotlib.pyplot as plt

GUARDAR_ARTEFACTOS = True # Cambia a False para desactivar la persistencia
```

```

if GUARDAR_ARTEFACTOS:
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    artifacts_dir = base_dir / "artifacts" / "tarea_2_1" / timestamp
    artifacts_dir.mkdir(parents=True, exist_ok=True)

    figuras_dir = base_dir / "docs" / "trabajo" / "figuras"
    figuras_dir.mkdir(parents=True, exist_ok=True)

    # Tablas resumen
    resumen_cv.to_csv(artifacts_dir / "resumen_cv.csv", index=False)
    resumen_test.to_csv(artifacts_dir / "resumen_test.csv", index=False)

    # Reporte del mejor modelo
    with open(artifacts_dir / f"reporte_{mejor_nombre}.txt", "w", encoding="utf-8") as f:
        f.write(reportes_clase[mejor_nombre])

    # Serializacion del mejor modelo entrenado
    dump(mejores_modelos[mejor_nombre], artifacts_dir / f"modelo_{mejor_nombre}.pkl")

    # Copia de la matriz de confusion para el informe final
    labels_ordenados = np.unique(y_test)
    fig_cm, ax_cm = plt.subplots(figsize=(8, 6))
    ConfusionMatrixDisplay(
        confusion_matrix=matrices_confusion[mejor_nombre],
        display_labels=labels_ordenados,
    ).plot(ax=ax_cm, xticks_rotation=45, colorbar=False)
    ax_cm.set_title(f"Matriz de confusion - {mejor_nombre}")
    fig_cm.tight_layout()

    ruta_figura = figuras_dir / f"matriz_confusion_{mejor_nombre}_{timestamp}.png"
    fig_cm.savefig(ruta_figura, dpi=300, bbox_inches="tight")
    plt.close(fig_cm)

    # Metadatos minimos para auditoria
    metadata = {
        "timestamp": timestamp,
        "dataset_fuente": fuente_datos,
        "random_state": RANDOM_STATE,
        "modelo_ganador": mejor_nombre,
        "ruta_figura_confusion": str(ruta_figura),
        "metricas_modelo_ganador": (
            resumen_test.loc[resumen_test["modelo"] == mejor_nombre]
            .iloc[0]
            .to_dict()
        ),
    }

    with open(artifacts_dir / "metadata.json", "w", encoding="utf-8") as f:
        json.dump(metadata, f, ensure_ascii=False, indent=2)

    print(f"Artefactos guardados en: {artifacts_dir}")
    print(f"Matriz de confusion guardada en: {ruta_figura}")
else:
    print("Persistencia desactivada. Cambia GUARDAR_ARTEFACTOS=True para guardar artefactos")

```

Artefactos guardados en: /mnt/vms/aprendizaje-automatico-relacional/artifacts/tarea_2_1/20260527_124833

Matriz de confusion guardada en: /mnt/vms/aprendizaje-automatico-relacional/docs/trabajo/figuras/matriz_confusion_ArbolDecision_20260527_124833.png

Conclusiones de la Tarea 2.1

Con esta implementación ya tienes un flujo completo y reproducible para modelos base en Cora:

1. Integración de información nativa y relacional.
2. Selección de hiperparámetros con validación cruzada estratificada.
3. Evaluación rigurosa en test con métricas globales y por clase.

Este resultado sirve como línea base sólida para comparar en tareas posteriores modelos más avanzados de aprendizaje sobre grafos.